Bolinho

Solution for data gathering, processing and interaction

Hefestus

Copyright © 2023 Hefestus.

Table of contents

1. Home 3
   - 1.1 Home 3
   - 1.2 Setup 4
2. Back End 6
   - 2.1 Back End 6
   - 2.2 DataHandler.py 7
   - 2.3 DBHandler 8
   - 2.4 DataHandler 11
3. Front End 12
   - 3.1 Front End 12
   - 3.2 Pages 13
   - 3.3 Components 16
   - 3.4 Contexts 19
   - 3.5 Alerts 20
   - 3.6 Styling 21
4. API 22
   - 4.1 API 22
   - 4.2 Data types 23
   - 4.3 Load and Save Data 31
   - 4.4 Commands 35
   - 4.5 Front-end exposed functions 37
5. About 39
   - 5.1 About 39

Copyright © 2023 Hefestus.

1. Home

1.1 Home

Documentation of the FullStack solution **bolinho**.

This documentation automatically generates a PDF file from it's content. You can download it here.

1.1.1 You can see the React Front-end HERE!

This is a static version of the app, without access to the server, therefore most features won't work.

> **Info**
>
> Remember that you need to build the app for it to show on the static page, so run `npm run buildWeb` or something similar to build it.

Use the **Tabs** above to navigate through the documentation.

1.1.2 Running

As for running the program we have a few options:

- Run only the frontend `npm run startWeb`
- Run only the backend `npm run startEel`
- Serve the full application `npm run serve`

  This command will start the eel as headless and start the web serve, it doesn't need to build the front end before executing. **Less performant**.
- Run the full application `npm run start`

  With this command it will first build the react front end, then run the python script.
- Build the react frontend `npm run buildWeb`
- Build binaries. `npm run buildBin`
- You can build the "binaries", more like a python environment wrapper, it uses PyInstaller to generate the bins.
- The output path is `bolinho/src/dist/`

> Did you like this documentation? You can check out the repo ZRafaF/ReadTheDocksBase for more info 😄.

Copyright © 2023 Hefestus.

1.2 Setup

This page will define the step-by-step to build this project.

This project assumes you have the latest version of Python, **PIP** and **GIT**,

This project was developed using python version `Python 3.10.x`

1.2.1 Clone the repo

Bash

```bash
git clone https://github.com/HefestusTec/bolinho

cd bolinho
```

1.2.2 Creating a virtual environment

The following step isn't mandatory but **recommended**.

Bash

```bash
python -m pip install --user virtualenv

python -m venv venv
```

The a directory `venv` should be created in the root folder.

How to activate:

Windows activation

```
venv/Scripts/activate
```

or

Linux activation

```bash
source venv/bin/activate
```


Copyright © 2023 Hefestus.

1.2.3 Installing dependencies

Bash

```bash
npm run installDep
```

1.2.4 Documentation

The following step is only required for those that want to **edit the documentation**.

Installing dependencies

Bash

```bash
pip install -r docs/requirements.txt
```

Build

We have two options to create a build:

- **Serve**:

  This option is used for debugging, it will open the static page in one of the localhost ports.

  ```bash
  mkdocs serve
  ```

- **Build**:

  This option creates a build of the documentation and saves it on de directory `/site/`.

  ```bash
  mkdocs build
  ```

> **Note**
>
> Be aware of the **Environment Variable** `ENABLE_PDF_EXPORT`, it will only generate the PDF if this variable is set to `1`.
>
> You can change the `mkdocs.yml` file and remove this line if you so choose.

For more info about the documentation please checkout ZRafaF/ReadTheDocksBase.

Copyright © 2023 Hefestus.

2. Back End

2.1 Back End

2.1.1 DBHandler.py

Defines database models using orm_sqlite

2.1.2 Models

- Material
- Experiment
- Body
- Reading

2.1.3 Classes

DBHandler

Handles database connection and CRUD operations

METHODS

- **init**(self, db_path)
- add_material(self, name, batch)
- add_experiment(self, name, material, date, time, load_loss_limit, max_load, max_travel, max_time, compress, z_axis_speed)
- add_reading(self, experiment, load, z_pos, time)
- get_materials(self)
- get_material_by_id(self, id)
- get_bodies(self)
- get_body_by_id(self, id)
- get_bodies_by_material(self, material)
- get_bodies_by_type(self, body_type)
- get_bodies_by_material_and_type(self, material, body_type)
- get_experiments(self)
- get_experiment_by_id(self, id)
- get_experiments_by_material(self, material)
- get_experiments_by_date(self, date)
- get_experiments_by_date_and_material(self, date, material)
- get_experiment_readings(self, experiment)
- delete_experiment_by_id(self, id)
- delete_material_by_id(self, id)
- populate(self)

Copyright © 2023 Hefestus.

2.2

Handles Raspberry Pi GPIO and data acquisition

2.2.1 Classes

DataHandler

Abstract class for data acquisition

LoadCell

Handles load cell data acquisition

StepMotor

Handles step motor data acquisition

Copyright © 2023 Hefestus.

2.3 DBHandler

2.3.1 DBHandler.py

Defines database models using orm_sqlite

2.3.2 Models

- Material
- id : Integer Primary Key
- name: String
- batch: String
- Experiment
- id : Integer Primary Key
- name: String
- material: Integer Foreign Key
- date: Date
- time: Time
- load_loss_limit: Float
- max_load: Float
- max_travel: Float
- max_time: Float
- compress: Boolean
- z_axis_speed: Float
- Body
- id : Integer Primary Key
- body_type: String
- material: Integer Foreign Key
- param_a: Float
- param_b: Float
- height: Float
- Reading
- id: Integer Primary Key
- experiment: Integer Foreign Key
- load: Float
- z_pos: Float
- time: Float

Copyright © 2023 Hefestus.

2.3.3 Classes

DBHandler

Handles database connection and CRUD operations

METHODS

- **init**(self, db_path)
- Creates database connection, binds models to database, creates and populates the database if it doesn't exist
- db_path: Path to database file
- add_material(self, name, batch)
- name: Name of the material
- batch: Batch number of the material
- add_experiment(self, name, material, date, time, load_loss_limit, max_load, max_travel, max_time, compress, z_axis_speed)
- name: Name of the experiment
- material: Name of the material
- date: Date of the experiment
- time: Time of the experiment
- load_loss_limit: Load loss limit of the experiment
- max_load: Maximum load of the experiment
- max_travel: Maximum travel of the experiment
- max_time: Maximum time of the experiment
- compress: Whether the experiment is compressive or not
- z_axis_speed: Speed of the z axis
- add_reading(self, experiment, load, z_pos, time)
- experiment: Name of the experiment
- load: Load of the reading
- z_pos: Z position of the reading
- time: Time of the reading
- get_materials(self)
- Returns all materials in the database
- get_material_by_id(self, id)
- id: Id of the material
- Returns the material with the given id
- get_bodies(self)
- Returns all bodies in the database
- get_body_by_id(self, id)
- id: Id of the body
- Returns the body with the given id

Copyright © 2023 Hefestus.

- get_bodies_by_material(self, material)
  - material: Name of the material
- Returns all bodies with the given material
- get_bodies_by_type(self, body_type)
  - body_type: Type of the body
- Returns all bodies with the given type
- get_bodies_by_material_and_type(self, material, body_type)
  - material: Name of the material
  - body_type: Type of the body
- Returns all bodies with the given material and type
- get_experiments(self)
- Returns all experiments in the database ordered by date and time descending
- get_experiment_by_id(self, id)
  - id: Id of the experiment
- Returns the experiment with the given id
- get_experiments_by_material(self, material)
  - material: Name of the material
- Returns all experiments with the given material
- get_experiments_by_date(self, date)
  - date: Date of the experiment
- Returns all experiments with the given date
- get_experiments_by_date_and_material(self, date, material)
  - date: Date of the experiment
  - material: Name of the material
- Returns all experiments with the given date and material
- get_experiment_readings(self, experiment)
  - experiment: Name of the experiment
- Returns all readings of the given experiment
- delete_experiment_by_id(self, id)
  - id: Id of the experiment
- Deletes the experiment with the given id and all its readings
- delete_material_by_id(self, id)
  - id: Id of the material
- Deletes the material with the given id and all its experiments and readings
- populate(self)
  - Populates the database with some dummy data

Copyright © 2023 Hefestus.

2.4 DataHandler

2.4.1 DataHandler.py

Handles Raspberry Pi GPIO and data acquisition

2.4.2 Classes

DataHandler

Abstract class for data acquisition

LoadCell

[NOT IMPLEMENTED]

Handles load cell data acquisition

StepMotor

[NOT IMPLEMENTED]

Handles step motor data acquisition

Copyright © 2023 Hefestus.

3. Front End

3.1 Front End

As front end framework/library we are using ReactJs.

You can check out the about page for a list of the **third party projects** being used.

Global variables

The global variables can be found at `variables.css`. You can import it into your stylesheet using

CSS

```
@import url(./variables.css);
```

3.1.1 Component architecture

```
flowchart TD
    App --> MainPage
    App --> ExperimentPage
    MainPage --> SideBar
    SideBar --> Header
    MainPage --> Content
    Content --> Início
    Content --> Calibrar
    Content --> Controlar
    Content --> Config.
    Content --> Sobre
```


Copyright © 2023 Hefestus.

3.2 Pages

The following pages are just place holders / mockups of the final application.

3.2.1 Initial page

Main

Initial page of the application, you are able to:

- Visualize previous experiments;
- Exclude experiments;
- Exclude materials;
- Access different sub-pages;
- Generate report; and more!.

Copyright © 2023 Hefestus.

Calibration sub-page

Control sub-page

Configuration sub-page

Copyright © 2023 Hefestus.

About sub-page

3.2.2 Experiment page

Main

Copyright © 2023 Hefestus.

3.3 Components

Most custom React components are here

3.3.1 ZoomComponent

Component that enables zooming on it's contents, press and hold to zoom in.

It uses the use-long-press rook to handle this action.

You can enable/disable and change the activation time on the `GlobalConfigContext` context.

Props

React

```
{
    scaleOrigin = "top",
    className = "",
    children,
}
```

- `scaleOrigin` : What is the origin point (which direction does it grows).
- `className` : ClassName of the component.
- `children` : Children props, Don't worry about it.

Usage example

React

```
import ZoomComponent from "../zoomComponent/zoomComponent";
import ExperimentsInspector from "./experimentsInspector/experimentsInspector";

export default function exampleComponent(){
    return(
        <ZoomComponent
            className={styleModule.experiments_inspector}
            scaleOrigin="bottom left"
        >
            <ExperimentsInspector />
        </ZoomComponent>
    );
}
```


Copyright © 2023 Hefestus.

3.3.2 ExperimentInspector

This component makes use of the `SelectedObjectsContext`.

The experiment inspector component holds the following children:

- `ColorPicker`: Color picker component for choosing the color of the active experiment.
- `ExperimentButton`: Buttons that make the list of selected experiments.
- `ExperimentDescription`: Component that parses the description of the active material.

Props

`none`

Usage example

React

```react
import { eel } from "./App";
import ExperimentsInspector from "./experimentsInspector/experimentsInspector";

export default function exampleComponent(){

    const getMaterialList = async () => {
        try {
            const materialList = JSON.parse(await eel.get_material_list()());
            return materialList;
        } catch (error) {
            return [];
        }
    };

    return(
        <SelectedObjectsContext.Provider>
            <ExperimentsInspector />
        </SelectedObjectsContext.Provider>

    );
}
```


Copyright © 2023 Hefestus.

3.3.3 MaterialSelector

The `MaterialSelector` component holds the following children:

- `MaterialSelectorButton` : Buttons that make the list of available experiments.
- `DropdownButton` : Buttons that make a dropdown, so the user can choose which experiment they want to inspect.

It can be wrapped with the `ZoomComponent` to allow zooming.

Props

React

```
{
    materialList // List of available materials, fetched from the backend
}
```

Usage example

React

```
import { eel } from "./App";
import MaterialSelector from "./materialSelector/materialSelector";

export default function exampleComponent(){

    const getMaterialList = async () => {
        try {
            const materialList = JSON.parse(await eel.get_material_list()());
            return materialList;
        } catch (error) {
            return [];
        }
    };

    return(
        <MaterialSelector materialList={getMaterialList} />
    );
}
```


Copyright © 2023 Hefestus.

3.4 Contexts

This page contains all the contexts used on the application.

3.4.1 GlobalConfigContext

React

```
{
    theme: "light", // light | dark
    animationSpeed: "slow", // fast | slow | off
    animateGraph: "on", // on | off
    enableZoom: true, // Should zooming be enable?
    zoomDelay: 300, // How long [ms] should I press to zoom
    blurOnZoom: true, // Should it blur when zooming?
    absoluteMaximumForce: 10000,
}
```

3.4.2 SelectedObjectsContext

This context is acessible to the children of the `MainPage` component. It holds a list of the selected objects "experiments data".

React

```
{
    material, // material_fragment
    experiment, // experiment_fragment
    data_array, // data_array_fragment,
    color, // color associated to an experiment
}
```


Copyright © 2023 Hefestus.

3.5 Alerts

This project is using react-toastify to handle alerts. If you want more in-depth info about them please refer to the documentation.

3.5.1 Usage

Here is an exemple of how you can create an alert.

React

```
import React from "react";
import { toast } from "react-toastify";

export default function myComponent() {
    const iWasPressed = () => {
        // This will show an error alert.
        toast.error("Não foi possível acessar o backend");
    };

    return(
        <button onClick={iWasPressed}>
            Press me :D
        </button>
    );
}
```


Copyright © 2023 Hefestus.

3.6 Styling

Here we will show some tips and styling techniques used on this project.

3.6.1 Linking stylesheets to a component

Stylesheets are made with `style.module.css` and imported into the `.jsx` or `.tsx` allowing us to use it as an object to name `className`s.

As for naming convention snake case is being used for styling names

React

```
import styleModule from "./mainPage.module.css";

<div className={styleModule.my_custom_div}>
    Im a styled div
<div>
```

3.6.2 Adding ellipsis

You can add ellipsis to most texts using the following code

CSS

```
white-space: nowrap;
overflow: hidden;
text-overflow: ellipsis;
```

Tip

This **must not be wrapped in a `flex` div**, therefore we recommend you using a div only for the text, for example:

React

```
<div className="my-cool-div">
    <div className="my-text">
            This text will add ellipsis when it overflows.
    </div>
</div>
```

CSS

```
.my-text{
    white-space: nowrap;
    overflow: hidden;
    text-overflow: ellipsis;
}
```


Copyright © 2023 Hefestus.

4. API

4.1 API

In this section you will be able to finde every **API call** available.

These `calls` are exposed to the **front-end** via the `eel` object, giving it access to the **data base**, **systems** and **hardware**. This solutions makes use of the eel library to realize the communication between the front-end and back-end;

This API reference will show the methods being called by the front-end in JavaScript, and every call should be made **asynchronously**.

Copyright © 2023 Hefestus.

4.2 Data types

All different data types will be shown in this page

4.2.1 DataPoint

Python

```python
class DataPoint:
    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y
```

- `x` : Position at the mesure moment
- type: `float`
- Unity: `mm`
- `y` : Force at the mesure moment
- Type: `float`
- Unity: `N`

4.2.2 DataPointArray

Python

```python
class DataPointArray:
    def __init__(self, id=0, data_array=[]):
        self.id = id
        self.data_array = data_array
```

- `id` : Identification (or key) of this element on the data base.
- type: `int`
- Unity: `mm`
- `data_array` : Array of data points, this is the "reading" of an experiment.
- Type: `[DataPoint...]`
- Unity: N/A

Copyright © 2023 Hefestus.

4.2.3 AutoStopParams

Python

```python
class AutoStopParams:
    def __init__(self, force_loss=20, max_force=1000, max_travel=100, max_time=600):
        self.force_loss = force_loss
        self.max_force = max_force
        self.max_travel = max_travel
        self.max_time = max_time
```

- `force_loss` : Max force loss to trigger auto-stop.
- Type: `float`
- Unity: `%`
- `max_force` : Max force limit to trigger auto-stop.
- Type: `float`
- Unity: `N`
- `max_travel` : Max distance the experiment head can travel during the experiment.
- Type: `float`
- Unity: `mm`
- `max_time` : Experiment time limit.
- Type: `float`
- Unity: `s`

Copyright © 2023 Hefestus.

4.2.4 BodyParams

Python

```python
class BodyParams:
    def __init__(self, type=0, param_a=0, param_b=0, height=0):
        # Body format | 1 = Rectangle | 2 = Cylinder | 3 = Tube
        self.type = type

        # Rectangle = length | Cylinder = External diameter | Tube = External diameter
        self.param_a = param_a

        # Rectangle = depth | Cylinder = NULL | Tube = Internal diameter
        self.param_b = param_b

        # Height of the test body
        self.height = height
```

- `type` : Body format
- 1 = Rectangle
- 2 = Cylinder
- 3 = Tube
- Type: `int`
- Unity: N/A
- `param_a` : Param 'a' of the body
- Rectangle = length
- Cylinder = External diameter
- Tube = External diameter
- Type: `float`
- Unity: `mm`
- `param_b` : Param 'b' of the body
- Rectangle = depth
- Cylinder = NULL
- Tube = Internal diameter
- Type: `float`
- Unity: `mm`
- `height` : Height of the test body
- Type: `float`
- Unity: `mm`

Copyright © 2023 Hefestus.

4.2.5 ExperimentParams

Python

```python
class ExperimentParams:
    def __init__(
        self,
        stop_params=AutoStopParams(),
        body_params=BodyParams(),
        compress=True,
        z_speed=5,
    ):
        self.stop_params = stop_params
        self.body_params = body_params
        self.compress = compress
        self.z_speed = z_speed
```

- `stop_params` : Auto stop parameters of the experiment.
- Type: `AutoStopParams`
- Unity: N/A
- `body_params` : Body parameters of the experiment.
- Type: `BodyParams`
- Unity: N/A
- `compress` : Dictates if the experiment head move up or down. true = compress | false = expand.
- Type: `bool`
- Unity: N/A
- `z_speed` : Z axis speed during the experiment.
- Type: `float`
- Unity: `mm/s`

Copyright © 2023 Hefestus.

4.2.6 Date

Python

```python
class Date:
    def __init__(
        self,
        day=1,
        month=1,
        year=2023,
    ):
        self.day = day
        self.month = month
        self.year = year
```

- `day` : Day.
- Type: `int`
- Unity: N/A
- `month` : Month.
- Type: `int`
- Unity: N/A
- `year` : Year.
- Type: `int`
- Unity: N/A

Copyright © 2023 Hefestus.

4.2.7 Experiment

Python

```python
class Experiment:
    def __init__(
        self,
        id=0,
        date=Date(),
        experiment_params=ExperimentParams(),
        data_array_id=0,
        extra_info="",
    ):
        self.experiment_params = experiment_params
        self.id = id
        self.data_array_id = data_array_id
        self.extra_info = extra_info
```

- `id` : id (or key) of the experiment on the data base.
- Type: `int`
- Unity: N/A
- `date` : Date of the experiment.
- Type: `Date`
- Unity: N/A
- `experiment_params` : Parameters of the experiment.
- Type: `ExperimentParams`
- Unity: N/A
- `data_array_id` : Identification (or key) of this experiment `DataPointArray` or "reading", on the data base
- Type: `int`
- Unity: N/A
- `extra_info` : Extra information about the experiment.
- Type: `String`
- Unity: N/A

Copyright © 2023 Hefestus.

4.2.8 Supplier

Python

```python
class Supplier:
    def __init__(self, name="NONE", email=""):
        self.name = name
        self.email = email
```

- `name` : Name of the material supplier.
- Type: `String`
- Unity: N/A
- `email` : E-mail of the supplier.
- Type: `String`
- Unity: N/A

Copyright © 2023 Hefestus.

4.2.9 Material

Python

```python
class Material:
    def __init__(
        self, id=0, name="NONE", batch=0, experiment_array=[], supplier=Supplier(), extra_info=""
    ):
        self.id = id
        self.name = name
        self.batch = batch
        # array of the ids of experiments with this material
        self.experiment_array = experiment_array
        self.supplier = supplier
        self.extra_info = extra_info
```

- `id` : Id (or key) of the material on the data base.
- Type: `int`
- Unity: N/A
- `name` : Name of the material.
- Type: `String`
- Unity: N/A
- `batch` : Batch of the material.
- Type: `int`
- Unity: N/A
- `experiment_array` : Array with the `ids` or `keys` of the experiments made with this material.
- Type: `[int...]`
- Unity: N/A
- `supplier` : Supplier of the material.
- Type: `Supplier`
- Unity: N/A
- `extra_info` : Extra information about the material.
- Type: `String`
- Unity: N/A

Copyright © 2023 Hefestus.

4.3 Load and Save Data

This page gathers all the API calls related to loading and saving data.

4.3.1 Pre-experiment params

get_auto_stop_params()

Returns the `autoStopParams` from the machine.

JavaScript

```javascript
let myStopParams = eel.get_auto_stop_params();
```

set_auto_stop_params()

Sets the equipments `autoStopParams`.

JavaScript

```javascript
const autoStopParams = {
    forceLossLimit: 20, // If theres a loss of force greater than 20% the experiment will be halted
    maxForce: 10000,    // Maximum force limit in Newtons
    maxTravel: 100,     // Maximum travel in mm
    maxTime: 600        // Maximum experiment time in seconds
};

eel.set_auto_stop_params(autoStopParams);
```

set_experiment_settings(settings)

Sets the `experimentSettings`.

JavaScript

```javascript
const experimentSettings = {
    compress: true, // (Should the experiment head move up or down?) true = compressing | false =
expanding
    zAxisSpeed: 5,  // Z axis speed during the experiment in mm/s
};

eel.set_experiment_settings(experimentSettings);
```


Copyright © 2023 Hefestus.

set_experiment_body_params(experiment_body_params)

Sets the `ExperimentBodyParams`;

JavaScript

```
const experimentBodyParams = new ExperimentBodyParams();

eel.set_experiment_body_params(experimentBodyParams);
```

4.3.2 Experiment Data

get_experiment_data(experiment_index)

Returns an `array` of `DataPoint` from an experiment.

JavaScript

```
experimentData = await eel.get_experiment_data(2);
```

get_new_experiment_data(experiment_index, data_index)

Returns an `array` of `DataPoint` from an experiment from an `data_index` onwards.

This method is used to fetch new data points, without needing to update the already received data.

JavaScript

```
newData = await eel.get_new_experiment_data(2, 354);
```

4.3.3 Equipment Params

get_calibration_params()

Get the calibration params.

JavaScript

```
calibrationParams = await eel.get_calibration_params();
```


Copyright © 2023 Hefestus.

get_equipment_params()

Get the equipment params.

JavaScript

```
calibrationParams = await eel.get_calibration_params();
```

set_equipment_params()

Set the equipment params.

JavaScript

```
eel.set_equipment_params( myEquipmentParams );
```

4.3.4 Manipulating the data base

get_material_data(material_index)

Gets the `MaterialData` of a material, returns `NULL` if the index doesn't exists.

JavaScript

```
materialDataAtIndex = await eel.get_material_data(2);
```

set_material_data(material, material_index)

Sets the `MaterialData` of a material is a specific index.

JavaScript

```
materialData = new MaterialData()

eel.set_material_data(materialData);
```


Copyright © 2023 Hefestus.

get_material_list(filter)

Finds and returns a `list` of `MaterialData` that can be filtered.

You can pass a `filter` to make a filtered search.

Returns `NULL` if theres no material that matches the filter.

JavaScript

```
materialsList = await eel.get_material_list(name="Madeira");
```

4.3.5 Global configuration

save_global_config(JSON)

Use this method to pass a JSON to the back-end, so it can be saved on the machine.

JavaScript

```
eel.save_global_config(myGlobalConfigJSON);
```

load_global_config()

Use this method to load the global config from the machine

JavaScript

```
globalConfigurations = await eel.load_global_config();
```


Copyright © 2023 Hefestus.

4.4 Commands

This page gathers all the API calls related to instructions to the equipment.

Eg. manual control, start tasks etc.

4.4.1 Manual Control

manual_control_head(dist)

Sends a **solicitation** to move the experiment head, this movement can be positive or negative.

If the solicitation is sent during a **automatic experiment** it will halt the experiment and wait for the restart command. este irá ser interrompido, e aguardará o comando para reiniciar

4.4.2 Calibration

start_z_axis_calibration()

Starts the calibration of the Z axis.

start_load_cell_calibration()

Starts the calibration of the load cell.

4.4.3 Experiment commands

start_auto_experiment()

Starts an automatic experiment.

pause_experiment()

Pauses the current experiment and awaits the user commands.

Copyright © 2023 Hefestus.

resume_experiment()

Resumes the current experiment.

end_experiment()

Finishes the current experiment.

Copyright © 2023 Hefestus.

4.5 Front-end exposed functions

This page will show front-end functions that are exposed to the backend.

Interaction backend -> front-end

> **Warning**
>
> The functions can only be called if they are available on the `web/build` directory, therefore if you make a change using `npm run serve` won't show it, you will need to rebuild the front end with `npm run buildWeb` or by using `npm run start`.

> **Note**
>
> These functions can only be called after eel is initiated with `eel.init()`.

4.5.1 How to create and expose functions to the backend

React

```
function myJsFunction(message){
    console.log(`Got this from the back end ${message}`)
}

// This line exposes the function to the back end, note the second argument, it is the name
that the back end needs to call
window.eel.expose(myJsFunction, "myJsFunction");
```

Python

```
try:
    eel.myJsFunction("IT'S WORKING")
except:
    pass
```


Copyright © 2023 Hefestus.

4.5.2 toastSuccess

Uses React-Toastify to create an success alert.

Front-end implementation

React

```
function toastSuccess(msg);

window.eel.expose(toastSuccess, "toastSuccess");
```

Usage example

Python

```
try:
    eel.toastSuccess("Success!")
except:
    pass
```

4.5.3 toastError

Uses React-Toastify to create an error alert.

Front-end implementation

React

```
function toastError(msg);

window.eel.expose(toastError, "toastError");
```

Usage example

Python

```
try:
    raise
except:
    try:
        eel.toastError("Exception raised")
    except:
        pass
```


Copyright © 2023 Hefestus.

5. About

5.1 About

This page will present extra info about the project.

5.1.1 Licenses

This software is licensed and distributed under the **GNU General Public License v3.0**

```
Copyright (C) 2023 Hefestus

Bolinho is free software: you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation, either version 3 of the License, or
(at your option) any later version.

Bolinho is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License
along with Bolinho.  If not, see <http://www.gnu.org/licenses/>.
```

Included third-party projects

- Python Eel - see license
- MkDocs - see license
- Material for MkDocs - see license
- MkDocs With PDF - see license
- MkDocs PDF Export Plugin - see license
- JSX Lexer - see license
- Roboto family of fonts - see license
- React - see license
- rc-slider - see license
- Chart JS - see license
- React Chart JS 2 - see license
- React Long Press Hook - see license
- React Colorful - see license
- React Toastify - see license
- Use Debounce - see license

Copyright © 2023 Hefestus.

- Use Long Press - see license
- React Tooltip - see license

Agradecemos do fundo do coração todos os autores dos diferentes projetos utilizados, **software livre** é liberdade, muito obrigado a todos.

Copyright © 2023 Hefestus.